Self-Checkout: Comparing Six Architectures

Evidence from load reports, code metrics, and a fault-injection test

Shilin · CS6510 Assignment 7 · October 26, 2026 · github.com/SASUKE40/CS6510-2026

1 Summary

I wrote the same self-checkout API six times in Rust. All six use Axum, Tokio, and SQLite (WAL mode, `synchronous=FULL`). The same Java load client tested all six on the same laptop. This report compares them with four sources of evidence: the 12 submitted load reports, static code metrics, a coupling and cohesion analyzer, and a new test that stops one component at a time.

- **At default load, speed did not increase after the first design.** The three single-process designs are within 4% of each other. Each network hop added latency. Checkout p50 increased from 1.6 ms (monolith) to 4.7 ms (Microservices) and 9.4 ms (event-based).
- **Only event-based scaled on one machine.** With 10 times more stations, the other five changed by 0.75× to 1.09×. One SQLite writer that syncs to disk on each commit limited them. Event-based uses group commit. Its throughput increased 1.69 times to 7,239 scans/s, the highest of all six.
- **Code size increased 5.8 times.** Mean complexity per function stayed at 3.4–4.1. The new code is coordination code: saga, outbox, retries, and the event bus.
- **Coupling moved. It did not go away.** Service-based shares 5 of 7 tables. Microservices adds 4 synchronous service-to-service dependencies. Event-based adds 6 topics with 16 event types.
- **Service-based had the best fault tolerance.** Checkout continued to work when basket or inventory stopped. In Microservices and event-based, the same failures stopped checkout. An event-based service needed 30–42 s to recover. All other processes recovered in 55–78 ms.

For this workload (one store, a synchronous API, 10–100 stations), **service-based** gave the best balance. Its shared database is its scaling limit and its single point of failure.

2 What I compared and how

Evidence sources.

1. **Load reports.** “Default” is 10 stations for 60 s. “Stress” is 100 stations for 120 s. Each run starts with fresh databases: 2,000 SKUs with 10,000 units each.
2. **Size and complexity.** `tokei` 15.0 and `rust-code-analysis-cli` 0.0.25, on production code only.
3. **Coupling and cohesion.** `code_metrics.py` builds the module graph and reads the SQL in each module.
4. **Faults.** `fault_injection.py` stops each component and sends a request to each endpoint.

The scripts and raw data are in `comparison/`. `figures.py` draws all diagrams.

Limits of the data. Each report is one run on a shared laptop. The Microservices README measured up to 45% variation between runs. Event-based is submitted with group commit (`CHECKOUT_MAX_BATCH=256`: up to 256 writes share one fsync) and two Kafka producers: saga messages are sent at once, and scans are sent in batches. The other five commit each write on its own. The client continues to scan items after they sell out. Thus 46–66% of checkouts are correct `409 INSUFFICIENT_STOCK` rejections. Latency percentiles include only successful requests. All 12 runs passed the post-run invariant checks (stock, basket totals, saga and outbox state). The top three popular items were the same in 10 of 12 runs.

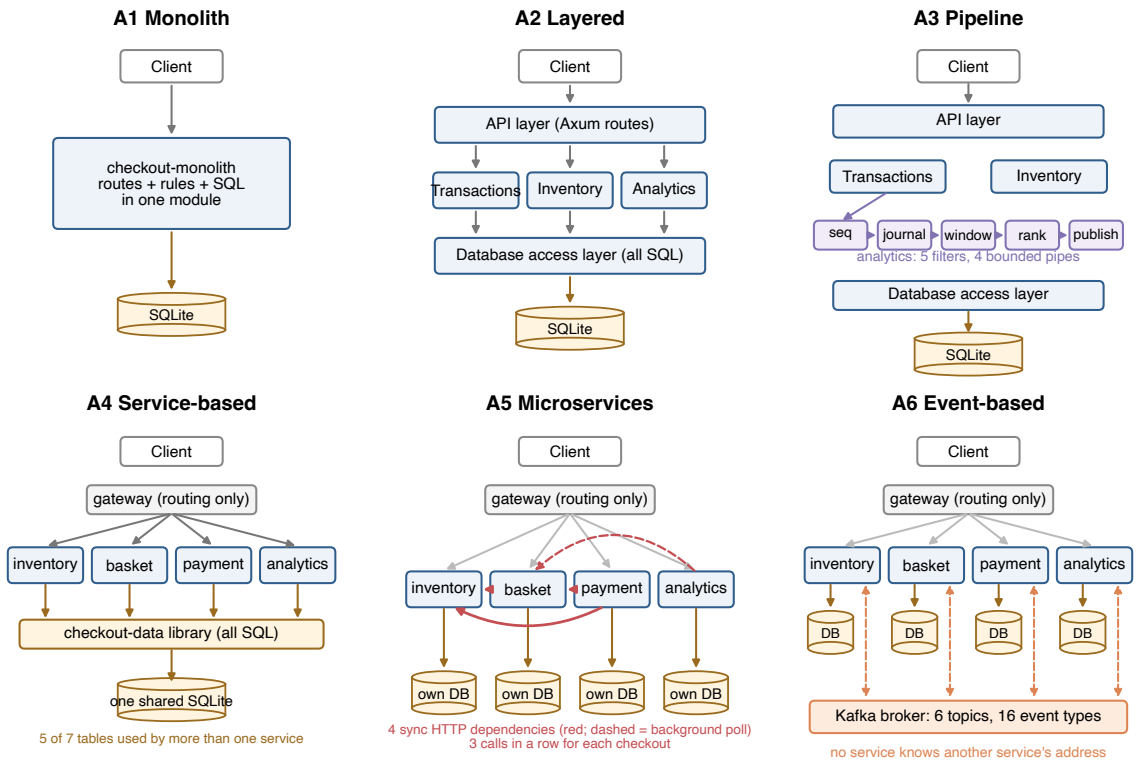


Figure 1: The six designs. All six have the same seven-route OpenAPI contract. Brown arrows are SQL access. Red arrows are synchronous service-to-service HTTP calls. Orange dashed arrows are Kafka events.

3 Performance

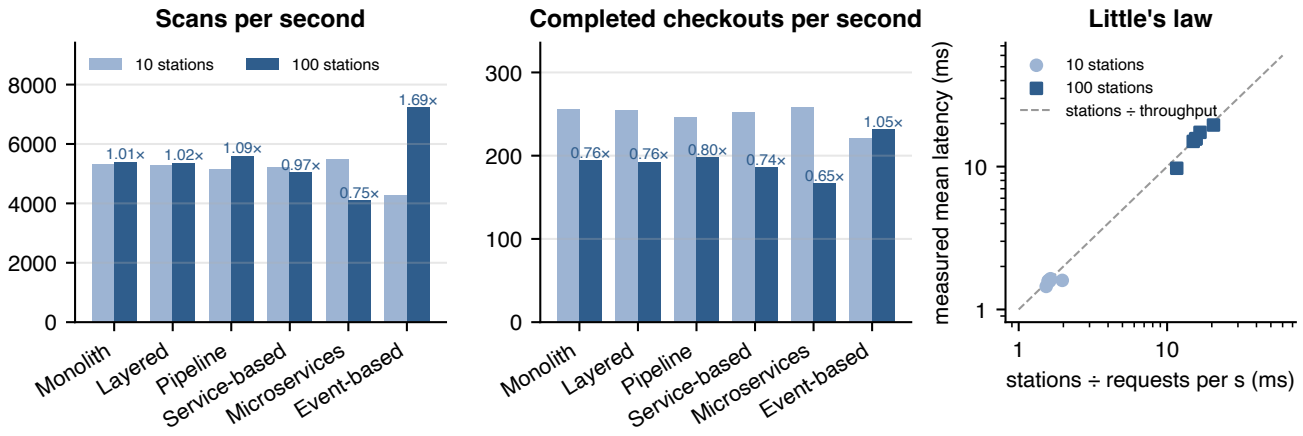


Figure 2: Left and centre: throughput. The label on each dark bar is the stress ÷ default ratio. Right: measured mean latency against stations ÷ throughput. Five designs are on the line. Event-based is below it: a rejected checkout needs 6 broker hops, so 46–66% of its checkouts are slow rejections, and the mean includes only successful requests.

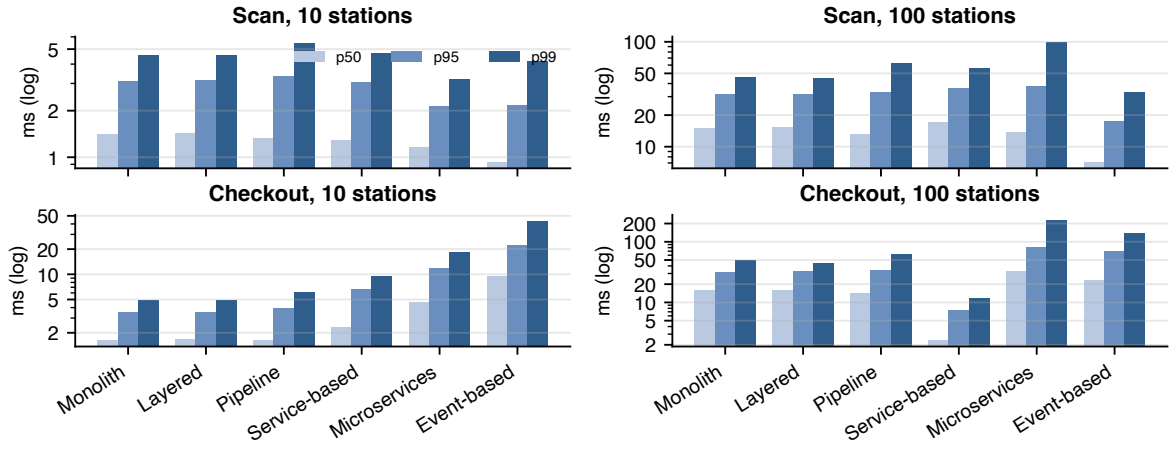


Figure 3: Latency percentiles for scan and checkout (log scale). At default load, each new hop increases the checkout tail. At stress, group commit gives event-based the lowest scan p99.

Layers and pipes had no cost. The monolith, layered, and pipeline designs are equal within the noise (Figure 2). In-process calls in Rust take nanoseconds. Thus the usual layered penalty did not occur. The pipeline moved analytics off the scan path. It kept one SQLite connection, so the bottleneck did not change.

The commit path set the limit for the other five designs. Each station is a closed loop with no wait time. Thus Little's law gives: mean latency = stations ÷ throughput. For the monolith at stress, 100 ÷ 6,427 requests/s = 15.56 ms. The measured mean is 15.58 ms. Ten times more stations gave ten times more latency and no more throughput. This shows one serial resource: one SQLite writer with an fsync for each commit.

A separate queue helped checkout in service-based. Its stress checkout p99 is 11.9 ms. The single-process designs have 45–63 ms. Payment has its own process. It does not wait in one queue behind starts and scans, which are 92% of all requests.

Each hop adds latency. The closed loop changes latency into lost throughput. A Microservices checkout makes 3 synchronous HTTP calls and 7 disk commits in 3 databases. Its stress p99 is 222 ms. An event-based checkout needs 4 broker hops (2 request-reply round trips) before the receipt. Its saga messages do not wait for a batch. At default load its p50 is 9.4 ms and its p99 is 43.6 ms. It has 78% of the Microservices throughput.

Group commit changed the result at stress. In event-based, the writes that wait at a service share one fsync. At default load the queue is short, so this does not help. At stress the queue is long. Throughput increased 1.69 times to 7,239 scans/s and 232 checkouts/s, the best of all six. Scan p99 (32.9 ms) is also the lowest at stress. Checkout p99 is still 137 ms, because the saga hops remain. Group commit is a configuration, not an architecture. But each of the four databases has its own write queue, so each one makes its own batches. One shared database has only one queue.

4 Code, coupling, and cohesion

Complexity per function stayed the same. The number of functions increased. Mean cyclomatic complexity is 3.4–4.1 in all designs (Figure 4). The event-based system has 355 functions. The monolith has 62. The step from service-based to Microservices added 1,024 lines and no new feature for the user. This code is the saga, its recovery, the outbox, a catalog copy, and an HTTP client with retries. The step to event-based added 752 more lines. Most of them are the event bus (455 lines) and the reply state machines. Functions with cyclomatic complexity over 10 increased from 4–6 to 21 and 31. `Payment::on_event` has the highest value of all six systems (40).

		Monolith	Layered	Pipeline	Service-based	Microservices	Event-based
SIZE	Production lines of code	499	756	1,120	1,108	2,132	2,884
	Test lines of code	318	356	429	558	1,030	1,158
	Functions and closures	62	99	125	125	248	355
	Deployable units	1	1	1	6	5	5
COMPLEXITY	Cyclomatic, mean per function	3.47	3.49	3.40	3.59	4.12	3.83
	Functions with cyclomatic > 10	4	4	5	6	21	31
	Maintainability index (0–100)	3.4	23.3	22.8	32.8	17.4	10.8
COUPLING	Propagation cost (file graph)	n/a	0.51	0.47	0.28	0.27	0.27
	Tables used by > 1 service	n/a	n/a	n/a	5	0	0
	Sync service-to-service calls	0	0	0	0	4	0
	Internal endpoints, pipes, or events	0	0	4	0	7	16
	Hops before a receipt	1	1	1	2	5	6
	Disk commits per checkout	1	1	1	1	7	7
COHESION	Business domains per SQL module	3.00	3.00	3.00	1.33	1.00	1.00
	Types with LCOM4 = 1	100%	60%	80%	67%	82%	85%

Figure 4: Static metrics on production code. Colour shows rank in each row: green is better, red is worse. Grey rows have no "better" direction. Propagation cost has no value for the monolith because it has only two files.

The maintainability index (MI) measures file size, not system simplicity. The monolith gets 3.4 because one 472-line file holds more than 90% of its code. Service-based gets the best value (32.8). Each of its services has only 79–121 lines, and one shared crate holds all the SQL.

Compile-time coupling decreased, then stopped. Propagation cost is the part of the system that a change to one file can reach. It decreased from 0.51 (layered) to 0.28 (service-based) and stayed there. No service imports the code of another service. Each service uses only shared library crates. In service-based, six deployables use the `checkout-data` crate. Thus one SQL change makes all six redeploy.

Service-based moved the coupling into the data. Five of its seven tables are used by more than one service. Payment has no tables of its own. It writes to the inventory and basket tables directly. This gives one ACID transaction for checkout without a saga. It also makes one schema change affect many services.

Database-per-service removed shared data and added runtime coupling. Microservices shares no tables. But a checkout now needs three calls in sequence: basket, inventory, and basket again. Event-based removes all service addresses. Services know only topic names. The coupling is now a contract: 6 topics, 16 event types, and at-least-once delivery for each handler.

Cohesion follows the partitioning. In the technical designs, one SQL module touches all three business domains. In the domain designs, each SQL module touches 1.0–1.33 domains. LCOM4 does not show this difference. The monolith `Store` gets a perfect LCOM4 of 1 because all methods share one connection. But it mixes catalog, basket, checkout, and analytics. The domain count shows this problem. LCOM4 does not.

Tests cost more with distribution. Test code increased 1.85 times from service-based to Microservices. The distributed suites start all services, and event-based also needs a real broker. A suite takes 3–11 s. In the single-process designs it takes less than 1 s.

5 Availability: one component stops

For each system, I started fresh databases (event-based with group commit and one Kafka producer). Then I stopped one component (SIGKILL, or `docker kill` for Kafka) and sent one request to each of the seven endpoints. Then I started the component again and measured the time until a full checkout worked.

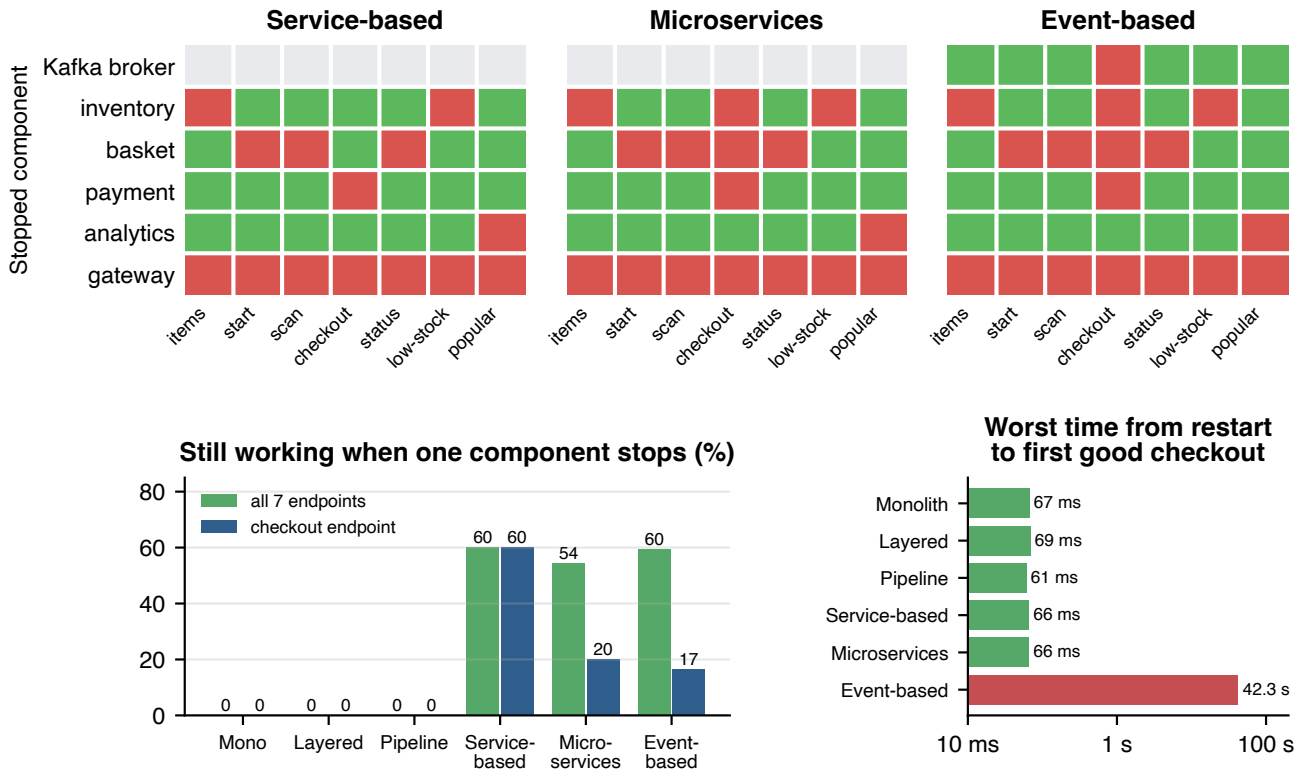


Figure 5: Top: endpoints that worked (green) or failed (red) while one component was stopped. Grey: the component does not exist. In the single-process designs, one failure stops all seven endpoints. Bottom: mean share of endpoints that worked, and the slowest recovery for each design.

- **Single-process designs lose all endpoints but recover quickly** (61–69 ms, the same as one service).
- **Service-based had the smallest effect on checkout.** Payment reads baskets and stock directly in the shared database. Thus checkout worked with basket or inventory stopped, and the test basket became `COMPLETED`. The shared database file is a single point of failure. This test did not stop it.
- **Microservices made checkout more fragile.** Checkout needs basket and inventory at runtime. If either one stops, checkout fails. It fails quickly (503 in 162 ms). The saga then sets the basket back to `OPEN`, and no stock is lost.
- **Event-based kept most endpoints but recovered slowly.** When Kafka stopped, 6 of 7 endpoints continued to work. But payment cannot send the first saga command, so checkout did not answer in the 15 s probe limit. A restarted service must join its consumer group again. Kafka removes the old member only after the 45 s session timeout. Thus recovery takes 30–42 s, about 500 times slower than the other designs.
- **The gateway is a single point of failure in all distributed designs.** It has no state, so you can replicate it.

6 Scalability and other trade-offs

- **Scalability.** Only event-based scaled on one laptop (Figure 2). Thus these runs **do not confirm** the high FoSA scalability rating for Microservices. The single-process and service-based designs have one write queue. More replicas cannot help, because SQLite lets only one writer work at a time. Microservices and event-based have one write queue for each service. You can also split basket by transaction ID and add Kafka partitions. I did not measure elasticity.
- **Consistency.** Monolith, layered, and service-based use one ACID transaction for each request. Microservices uses a saga. The pipeline can lose accepted scans in a crash.

- **Deployability and evolvability.** In service-based, one schema change redeploys all 6 units. Microservices and event-based each own their schema. Event-based is the easiest to extend: a new consumer of `basket.scans` (for example, fraud detection) needs no change to basket.
- **Operational cost.** The monolith is one binary and one file. Microservices is five processes and four databases. Event-based adds a Kafka broker. A Kafka 4.1.0 bug also stopped the basket consumer once (now Kafka 4.3.2, and a consumer joins its group again after an error).

7 FoSA architecture characteristics chart

The stars rate each design from 1 to 5 against the other five. The grey number is the rating in *Fundamentals of Software Architecture* (Richards & Ford, 2020). I added recoverability because the fault test showed it clearly.

Characteristic	Monolith	Layered	Pipeline	Service-based	Micro-services	Event-based
Partitioning	none	technical	technical	domain	domain	domain*
Quanta	1	1	1	1	2	4*
Simplicity	★★★★★ (–)	★★★★★ (5)	★★★★☆ (5)	★★★☆☆ (3)	★★☆☆☆ (1)	★☆☆☆☆ (1)
Overall cost	★★★★★ (–)	★★★★★ (5)	★★★★★ (5)	★★★★☆ (4)	★★☆☆☆ (1)	★☆☆☆☆ (3)
Performance	★★★★★ (–)	★★★★★ (2)	★★★★★ (2)	★★★★☆ (3)	★★★☆☆ (2)	★★★☆☆ (5)
Scalability	★☆☆☆☆ (–)	★☆☆☆☆ (1)	★☆☆☆☆ (1)	★★☆☆☆ (3)	★★★☆☆ (5)	★★★★☆ (5)
Fault tolerance	★☆☆☆☆ (–)	★☆☆☆☆ (1)	★☆☆☆☆ (1)	★★★★☆ (4)	★★★☆☆ (4)	★★★☆☆ (5)
Recoverability (not FoSA)	★★★★★ (–)	★★★★★ (–)	★★★★★ (–)	★★★★★ (–)	★★★★★ (–)	★★☆☆☆ (–)
Reliability / consistency	★★★★★ (–)	★★★★★ (3)	★★★★☆ (3)	★★★★★ (4)	★★★★☆ (4)	★★★☆☆ (3)
Modularity	★☆☆☆☆ (–)	★★☆☆☆ (1)	★★★☆☆ (3)	★★★☆☆ (4)	★★★★★ (5)	★★★★★ (4)
Deployability	★☆☆☆☆ (–)	★☆☆☆☆ (1)	★☆☆☆☆ (2)	★★★☆☆ (4)	★★★★☆ (4)	★★★★☆ (3)
Evolvability	★☆☆☆☆ (–)	★★☆☆☆ (1)	★★★☆☆ (3)	★★★☆☆ (3)	★★★★☆ (5)	★★★★★ (5)
Testability	★★★★★ (–)	★★★★★ (2)	★★★★★ (3)	★★★★☆ (4)	★★★☆☆ (4)	★★☆☆☆ (2)

Table 1: Evidence-based ratings (stars), with the FoSA book rating in grey. (–) means the book has no rating. *The book calls event-driven technically partitioned. Event-based has 4 quanta, but checkout acts as 1. Service-based has one quantum because of the shared database. Microservices has two, because checkout calls are synchronous.

Where the evidence does not agree with the book:

1. **Event-driven performance (book 5, measured 3).** It has the highest stress throughput. But the checkout contract is synchronous, so each event becomes request-reply. At default load it is the slowest design.
2. **Layered performance and testability (book 2, measured 5).** The book assumes separate tiers or a heavy stack. In-process Rust layers added no measurable latency, and the full test suite runs in less than 1 s.
3. **Fault tolerance, event-driven (book 5) and Microservices (book 4), measured 3.** Checkout needs basket and inventory.
4. **Scalability, Microservices (book 5, measured 3) and event-driven (book 5, measured 4).** Only event-based used the extra capacity on one host.

8 Conclusion

Each new architecture moved complexity to a new place. None of them removed it. On this evidence, service-based is the best overall: throughput equal to the monolith, the best stress checkout p99 (11.9 ms), and the best fault tolerance. Event-based has the highest stress throughput, but the slowest default latency and the slowest recovery. Microservices and event-based give data ownership and independent schemas. They cost 1.9–2.6 times the code and more checkout latency. This cost is correct only for many teams or for more than one database writer. For one store, it was higher than the benefit.